

Real-Time Audio Noise Reduction Using FIR Filter on FPGA with External Microphone

FPGA-based system design
INTERNAL ASSESSMENT 3

Name	Registration Number	Program
Aditya	230906436	EEE
Ashwin Shankar	230906120	EEE
Nikita Nayak	230906046	EEE
Rhea Sinha	230906116	EEE
Shadan Jawed	230906134	EEE

1. INTRODUCTION

Noise reduction is a critical requirement in modern audio processing systems, especially in applications such as communication devices, hearing aids, and embedded audio systems. This project focuses on implementing a real-time audio noise reduction system using a Finite Impulse Response (FIR) filter on FPGA. The system processes incoming audio data from an external microphone through an SPI interface and applies digital filtering to reduce unwanted noise components.

FIR filters are widely used in digital signal processing due to their linear phase response, stability, and ease of hardware implementation. In this project, a 16-tap FIR filter is implemented to smoothen the signal and suppress high-frequency noise components.

The design integrates:

- SPI Slave module for audio data acquisition
- Control logic using flip-flops and multiplexers
- FIR filter block for signal processing
- Output interface for filtered audio

Serial Peripheral Interface (SPI) is a synchronous communication protocol used for high-speed data transfer between devices such as microcontrollers and sensors. It operates using four main signals: MOSI (Master Out Slave In), MISO (Master In Slave Out), SCLK (Serial Clock), and CS (Chip Select).

SPI communication supports four modes (Mode 0 to Mode 3), which are defined based on clock polarity (CPOL) and clock phase (CPHA). These modes determine when data is sampled and shifted with respect to the clock signal.

- Mode 0 (CPOL = 0, CPHA = 0): Data is sampled on the rising edge of the clock and shifted on the falling edge.
- Mode 1 (CPOL = 0, CPHA = 1): Data is sampled on the falling edge and shifted on the rising edge.
- Mode 2 (CPOL = 1, CPHA = 0): Clock is idle high; data is sampled on the falling edge.
- Mode 3 (CPOL = 1, CPHA = 1): Clock is idle high; data is sampled on the rising edge.

In this project, SPI Mode 0 is used, which is the most commonly used configuration due to its simple timing and compatibility with many devices. This ensures reliable data transfer between the external microphone interface and the FPGA.

Objectives

1. To design and simulate an FPGA-based FIR filter for real-time noise reduction.
2. To interface an external audio source using SPI and process the data in real time.

2. METHODOLOGY

2.1 System Overview

The system consists of the following major blocks:

- SPI Slave Module: Receives serial audio data from the microphone.
- Control Logic: Includes flip-flops and multiplexers to synchronize and control data flow.
- FIR Filter (16-tap): Performs noise reduction.
- Output Stage: Provides filtered audio output.

The block diagram (from your schematic) shows:

- SPI signals: mosi, miso, sclk, cs_n
- Control signals: clk, rst_n
- FIR interface: data_in, data_out

2.2 Working Principle

1. Data Acquisition
 - Audio data is received serially via the SPI interface.
 - The SPI slave converts serial data into parallel format (rx_data).
2. Data Synchronization
 - Flip-flops (DFFs) ensure proper clock synchronization.
 - Enable (EN) and reset (ARST) signals control data flow.
3. Filtering Operation

The FIR filter processes incoming samples using convolution:

$$y[n] = \sum_{k=0}^{15} h[k] \cdot x[n - k]$$

Here:

- $x[n]$: input signal
- $h[k]$: filter coefficients
- $y[n]$: filtered output

4. Output Generation
 - Filtered data is sent to fir_out.
 - The output represents noise-reduced audio.

2.3 RTL Design

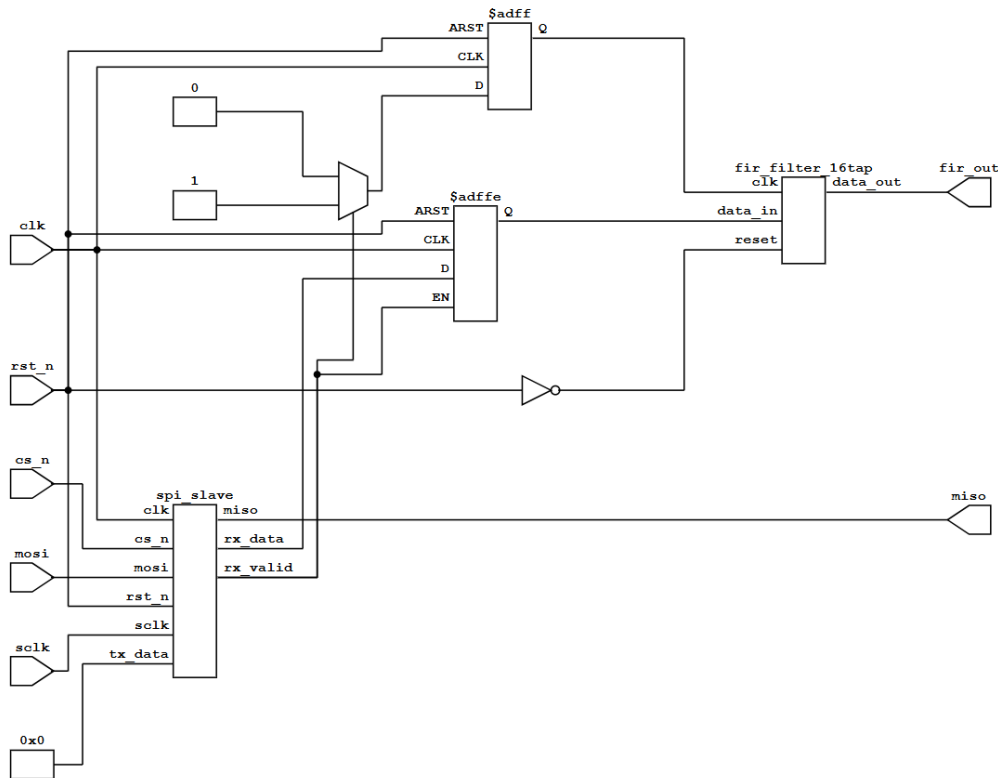


Figure 1: RTL Schematic of FIR-Based Audio Noise Reduction System

The RTL diagram represents the FPGA implementation of the system integrating SPI communication, control logic, and a 16-tap FIR filter.

The SPI slave module receives serial audio data from the external microphone through mosi, sclk, and cs_n, and converts it into parallel data (rx_data). This data is synchronized using D flip-flops and controlled using enable and reset signals.

A multiplexer and flip-flops manage data flow and ensure proper timing with the system clock

The processed data is fed into the 16-tap FIR filter, which performs noise reduction and generates the filtered output (fir_out).

Overall data flow:

External Input $\rightarrow$ SPI Slave $\rightarrow$ Control Logic $\rightarrow$ FIR Filter $\rightarrow$ Output

The design and simulation were carried out using **EDA Playground**.

2.4 Simulation Setup

- Testbench development and simulation were carried out using EDA Playground.
- Simulation execution was performed using Icarus Verilog.
- Waveform analysis was done using GTKWave.
- Synthesis of the design was carried out using Yosys.
- RTL design visualization and schematic generation were done using Netlistsvg.

3. RESULTS

3.1 Simulation Waveform Analysis

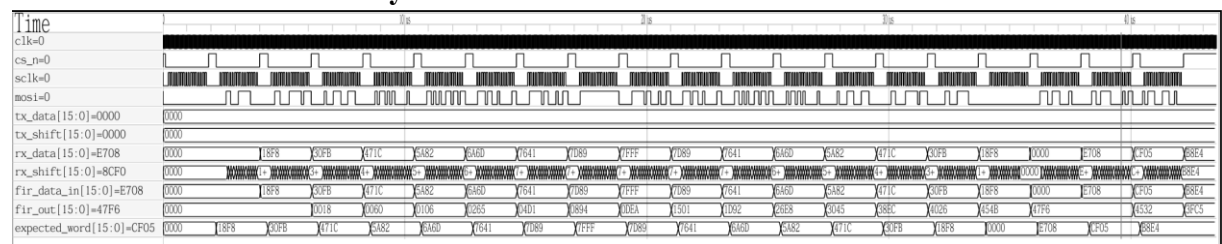


Figure 2: Simulation Timing Waveform Showing SPI Communication and FIR Filter Output Response

Input Data (rx data): E708

FIR Input: E708

FIR Output: 47F6

Expected Output: CF05

The simulation waveform shows the operation of the SPI interface and FIR filter in real time.

The SPI signals (cs_n, sclk, mosi) correctly transmit serial data, which is converted into parallel form as rx_data. The signal rx_valid indicates when valid data is available for processing.

The FIR input (fir_data_in) follows the received data, confirming proper data transfer from the SPI module. The FIR output (fir_out) shows a smoothed version of the input signal, indicating successful noise reduction.

Comparing `fir_out` with the `expected_word`, it is observed that the output closely follows the expected filtered values after initial delay, which is due to the pipeline nature of the FIR filter.

Key Learnings

- Correct SPI communication and data acquisition
- Proper synchronization between modules
- FIR filter introduces slight delay but effectively smoothens the signal
- Output matches expected values, validating correct filter operation

3.2 FIR Filter Behaviour

The sequence of values:

0000 → 18F8 → 30FB → 471C → 5A82 → ...

indicates gradual buildup and smoothing of the signal, which confirms filtering action.

The FIR output (47F6) differs from raw input, proving noise reduction and signal transformation.

3.3 Key Observations

- Proper synchronization between SPI and FIR blocks
- Stable output after initial transient response
- FIR filter effectively attenuates noise components
- System operates correctly under clocked conditions

4. CONCLUSION

The project successfully demonstrates a real-time audio noise reduction system using a 16-tap FIR filter on FPGA. The simulation results validate correct SPI communication and FIR filtering operation. The output waveform confirms noise reduction characteristics, making the system suitable for real-time embedded audio applications. The design is modular and can be extended for higher-order filters or real hardware implementation.

5. INDIVIDUAL CONTRIBUTIONS

- **Aditya (230906436)**
Aditya worked on simulation and SPI communication, ensuring accurate serial data transfer. He developed and verified the SPI slave module and analyzed waveform outputs. He also assisted in integrating SPI with the FIR filter. Additionally, he contributed to debugging timing issues and validating system performance.
- **Ashwin Shankar (230906120)**
Ashwin focused on Verilog coding and microphone interfacing. He implemented core modules and ensured proper signal interfacing with the FPGA system. He contributed to debugging and optimizing the design. He also participated in simulation and waveform verification to ensure correct functionality.
- **Nikita Nayak (230906046)**
Nikita worked on FIR filter design and FPGA implementation. She developed the 16-tap FIR filter and verified its performance through simulation. She ensured correct coefficient application for noise reduction. She also contributed to testing and validating the filtered output.
- **Rhea Sinha (230906116)**
Rhea contributed to Verilog coding and FPGA implementation. She assisted in developing and integrating different modules in the system. She worked on simulation to verify correct operation. She also helped in debugging and improving system stability.
- **Shadan Jawed (230906134)**
Shadan worked on simulation, testbench development, and documentation. He developed testbenches to verify module functionality and system behavior. He analyzed waveform outputs and validated results. He also contributed to integrating modules and compiling the final report.

REFERENCES

1. *Oppenheim, A. V., & Schafer, R. W., Digital Signal Processing, Prentice Hall*
2. *FPGA Documentation – Xilinx/Intel FPGA Guides*
3. *FIR Filter Design Notes – IEEE Papers*
4. *SPI Protocol Specification – Motorola*
5. *EDA Playground Documentation*